The Legal Game

Multi-Agent Legal Debate System Architecture

System Documentation

March 8, 2026

Abstract

The Legal Game is a sophisticated multi-agent NLP application designed to simulate legal-style debates over a motion. Built around a constrained debate corpus, the system eschews pure end-to-end generation in favor of a hybrid approach. It ensures reliability even without massive labeled datasets by intelligently combining deterministic preprocessing, retrieval augmented generation, weak supervision, structured argument composition, and explainable judging. This transforms the project from a simple text-generation demonstration into a robust argumentation engine.

Contents

1 Application Workflow & Architecture **2**
1.1 System Layers Diagram 2

2 Models and Tokenizers **2**
2.1 Prosecutor Model (Transformer Generation) 3
2.2 Defense Model (Retrieval Engine) 3

3 Mathematical Framework & Weak Supervision **3**
3.1 Retrieval Logic for Defense 3
3.2 Weak Stance Labels 4

4 The Explainable Judge **4**

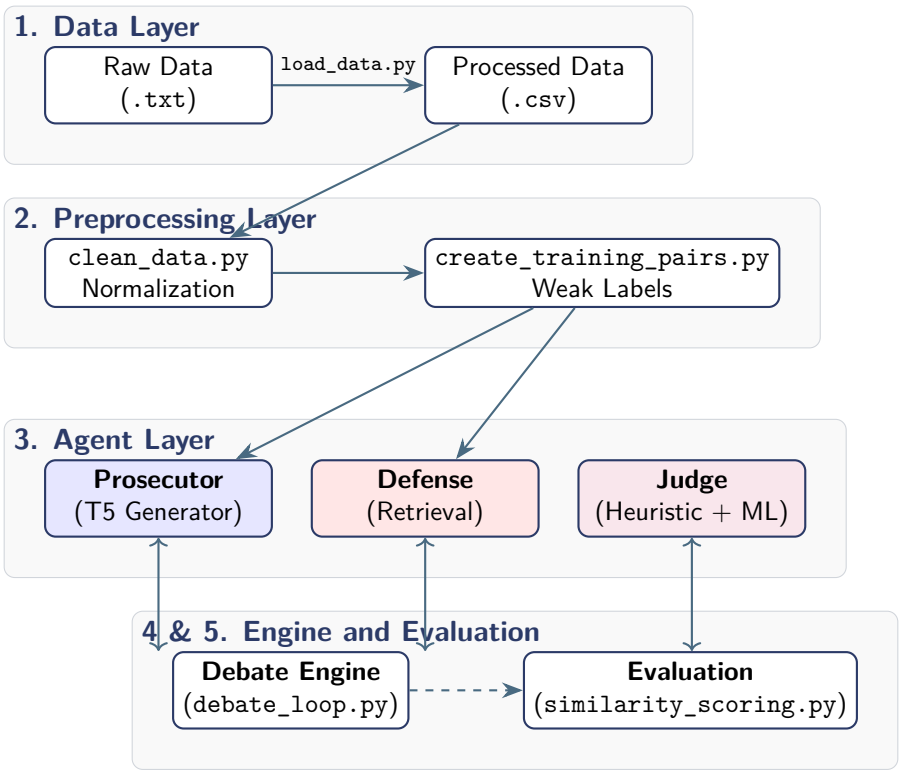
1 Application Workflow & Architecture

The application systematically evaluates a debate topic by orchestrating multiple specialized agents. Rather than depending on a single monolithic model, it executes the following logic loop:

1. **Data Retrieval:** Pulls relevant claims and evidence from the preprocessed dataset based on the input topic.
2. **Prosecutor Generation:** Composes a supporting argument for the motion.
3. **Defense Rebuttal:** Retrieves an opposing position and composes a grounded counter-argument.
4. **Adjudication:** Evaluates both sides using an explainable, multi-metric judge.
5. **Iterative Looping:** Runs the exchange for configured rounds and aggregates the final verdict.

1.1 System Layers Diagram

To achieve this, the architecture is divided into five strictly defined operational layers:



2 Models and Tokenizers

This project distinguishes itself by using heterogeneous modeling logic: a sequence-to-sequence transformer for generation, combined with deterministic retrieval and heuristic scoring for evaluation.

2.1 Prosecutor Model (Transformer Generation)

The Prosecutor is the sole component utilizing free-form generation. It is built on `google/flan-t5-small` using the `T5ForConditionalGeneration` inference class.

Prompt Design & Inference: The model uses a strict prompting format to map evidence to arguments:

```
Topic: <topic> Evidence: <evidence_text> Generate an argument.
```

At inference, it employs Beam Search (`num_beams=5`, `max_length=64`) rather than greedy decoding. This ensures grammatical stability and coherence.

Technical Dependency Warning: The `T5Tokenizer` inherently depends on the `sentencepiece` library. If `sentencepiece` is missing from the environment, the prosecutor model cannot be loaded, even if checkpoint files are present. The system handles this gracefully by falling back to template-based argument generation to prevent runtime crashes.

2.2 Defense Model (Retrieval Engine)

Because the available dataset is constrained, training a second free-form generator for the defense risks high hallucination rates and repetitive outputs. Instead, the defense operates as a retrieval-backed component. It loads the cleaned dataset, ranks candidate claims using lexical relevance, and formats a structurally grounded rebuttal based on actual evidentiary text.

3 Mathematical Framework & Weak Supervision

The core of the system's evaluation and retrieval relies heavily on **Lexical Jaccard Similarity**. For any two sets of lowercase word tokens, A and B, the Jaccard index is computed as:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

If either text resolves to zero tokens, the score defaults to 0.0.

3.1 Retrieval Logic for Defense

The defense evaluates and ranks candidate records (`rank_records`) using a weighted formula designed to balance relevance and novelty:

Defense Selection Heuristic

$$\text{Score} = 0.45(O_{\text{topic}}) + 0.35(1 - O_{\text{query}}) + 0.20(W_{\text{evidence}})$$

Rationale for weights:

- O_{topic} (Topic Overlap): Strongly prefers candidates conceptually close to the debate motion.
- O_{query} (Query Overlap): By subtracting this from 1, the system specifically hunts for counter-arguments that differ lexically from the Prosecutor's generated text.
- W_{evidence} (Evidence Priors): Assigns deterministic weights based on evidence quality (`[STUDY]` $\rightarrow$ 1.0, `[STATISTICS]` $\rightarrow$ 0.95, `[EXPERT]` $\rightarrow$ 0.9, `[ANALYSIS]` $\rightarrow$ 0.85, Unknown $\rightarrow$ 0.8).

3.2 Weak Stance Labels

To bootstrap the pipeline without thousands of hours of human annotation, the system generates weak supervision labels (`create_training_pairs.py`). It assumes that if a claim heavily borrows language from a piece of evidence, it supports it.

```
overlap = Jaccard(claim_tokens, evidence_tokens)
if overlap >= 0.08:
    stance = "support"
else:
    stance = "oppose"
```

4 The Explainable Judge

The Judge features two layers: a deterministic heuristic scorer and an optional Machine Learning classifier (`LogisticRegression` via `scikit-learn`). The deterministic layer is fully explainable and computes five distinct proxies for argument quality.

1. **Relevance:** Lexical overlap between topic and argument. $Rel = J(\text{topic}, \text{argument})$.
2. **Coherence (Length Proxy):** A structural proxy assuming longer, multi-sentence outputs correlate with coherent thought formulation.

$$Coh = \min\left(1.0, \frac{\text{sentences}}{3.0}\right)$$

3. **Specificity (Density Proxy):** A proxy for informational density, rewarding the use of longer, more complex vocabulary.

$$Spec = \min\left(1.0, \frac{\text{tokens with length } \geq 6}{\max(8, \text{total tokens})}\right)$$

4. **Evidence Coverage:** Rewards explicit academic cue words (e.g., *research, data, expert*).

$$Evid = \min\left(1.0, \frac{\text{matched cue terms}}{3.0}\right)$$

5. **Novelty:** Penalizes sides that simply repeat their opponent's wording.

$$Nov = \max\left(0.0, 1.0 - J(\text{opponent_arg}, \text{arg})\right)$$

Final Weighted Verdict

The deterministic judge aggregates these proxies into a final score:

$$\text{Total} = 0.35(Rel) + 0.25(Evid) + 0.15(Coh) + 0.10(Spec) + 0.15(Nov)$$

If the optional ML judge is active (`models/judge_model/judge_model.joblib`), it attempts to predict the winner by evaluating the resulting heuristic feature matrix (`pro_total`, `def_relevance`, etc.). Because it is trained on the heuristic labels, it currently learns to mimic the mathematical judge rather than surpassing gold-standard human annotations.